

Rapport Final

Évolution de l'Organisation du Code

L'architecture principale est restée stable entre le rendu2 et le rendu3. Les améliorations clés incluent :

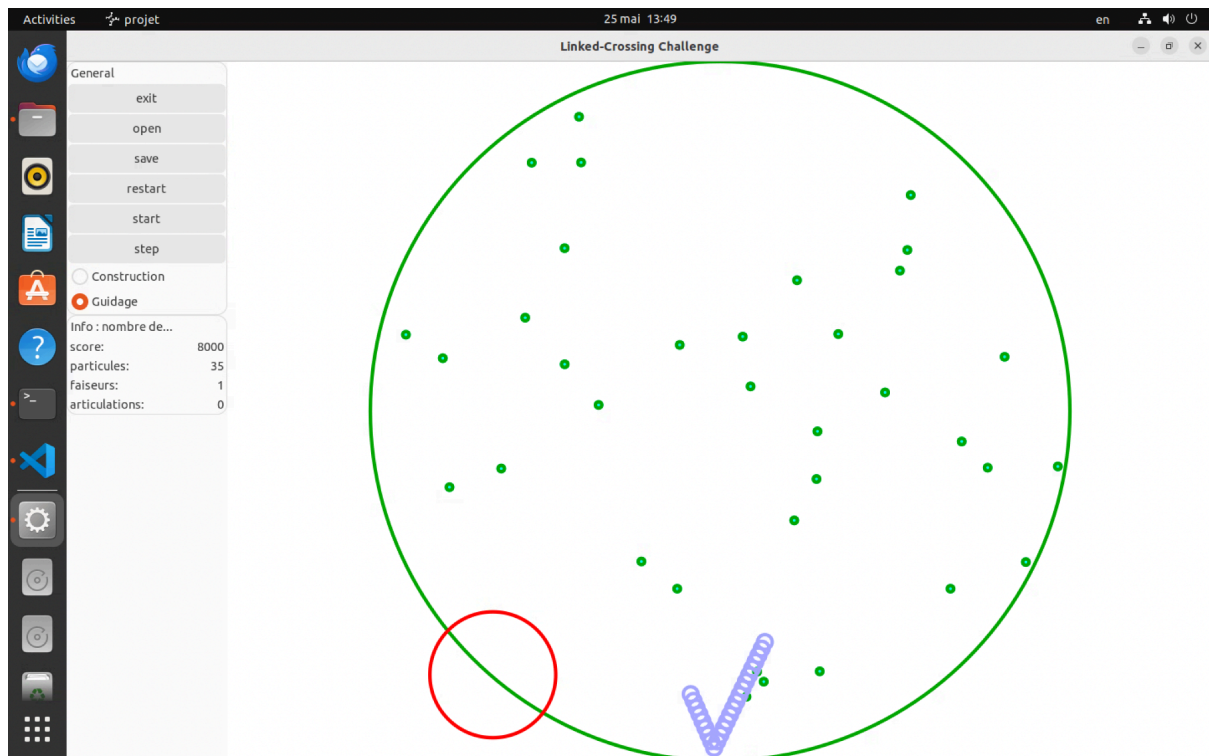
- Algorithme de Guidage de Chaîne : Implémentation de FABRIK (cinématique inverse) dans `chaîne.cpp` avec validation des limites de l'arène
- Système de Collision : Amélioration de la détection de collision articulation-faiseur pendant les phases de construction et de guidage
- Interaction Souris : Développement des mécaniques de capture de particules et définition d'objectifs intermédiaires via clics gauche/droit avec retour visuel en temps réel
- Logique d'État du Jeu : Ajout de la gestion complète des conditions de victoire/défaite avec synchronisation UI appropriée

La hiérarchie polymorphe `Mobile` et la séparation modulaire entre logique de jeu (`jeu.cpp`), entités (`mobile.cpp`), et mécanique de chaîne (`chaîne.cpp`) s'est révélée robuste tout au long du développement.

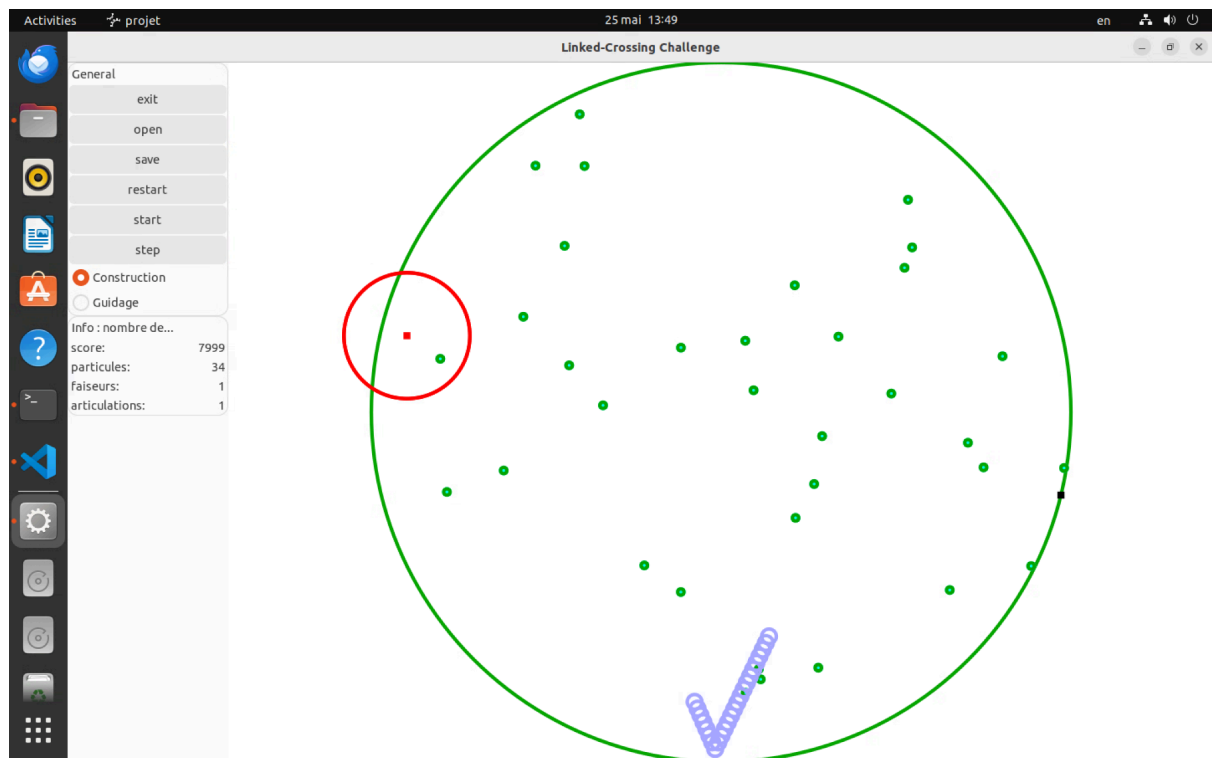
Exécution du Programme

t22.txt : Construction et Guidage de Chaîne

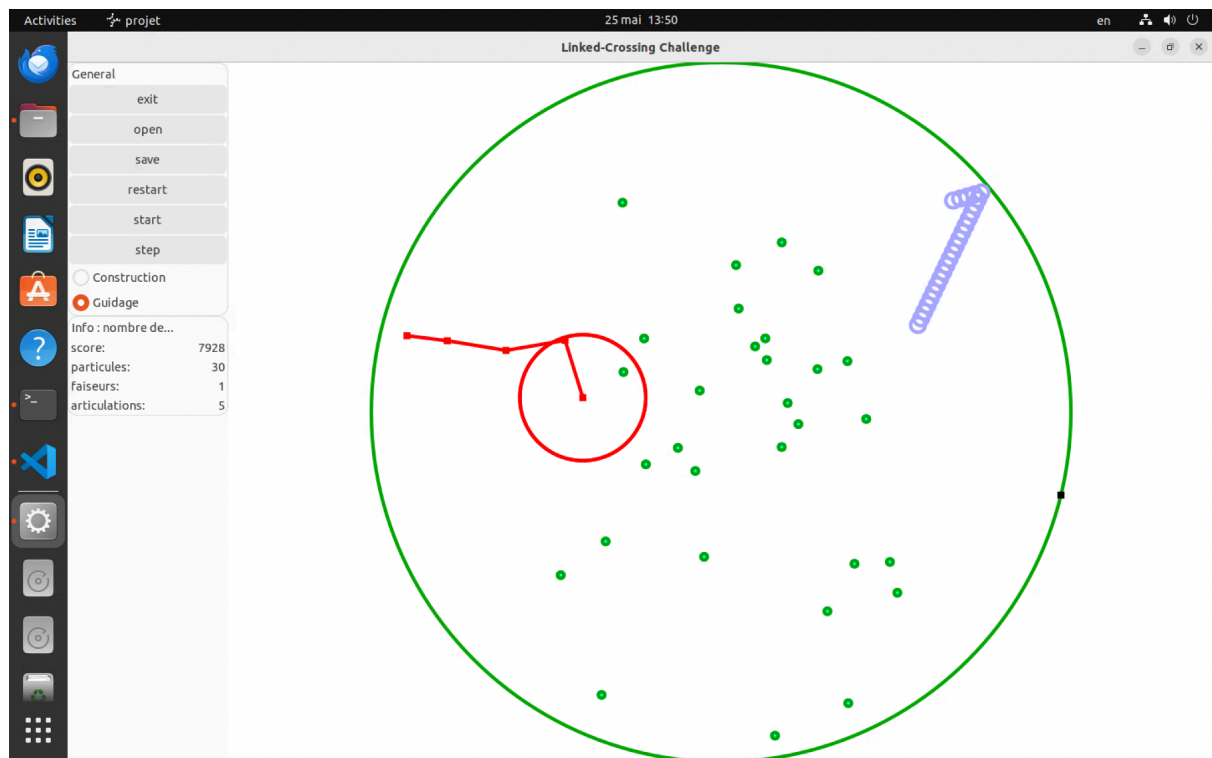
[Image 1 : État initial]



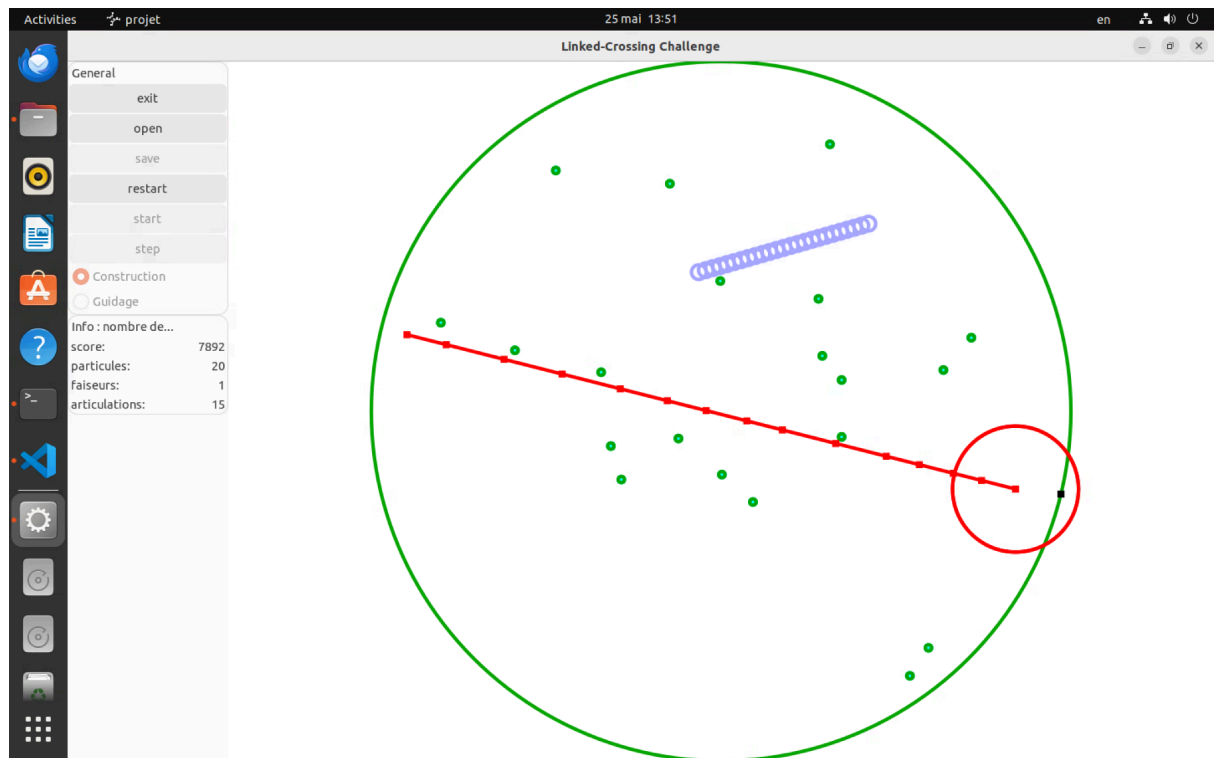
[Image 2 : Première articulation - établissement de la racine]



[Image 3 : Chaîne multi-articulation avec utilisation du mode guidage]



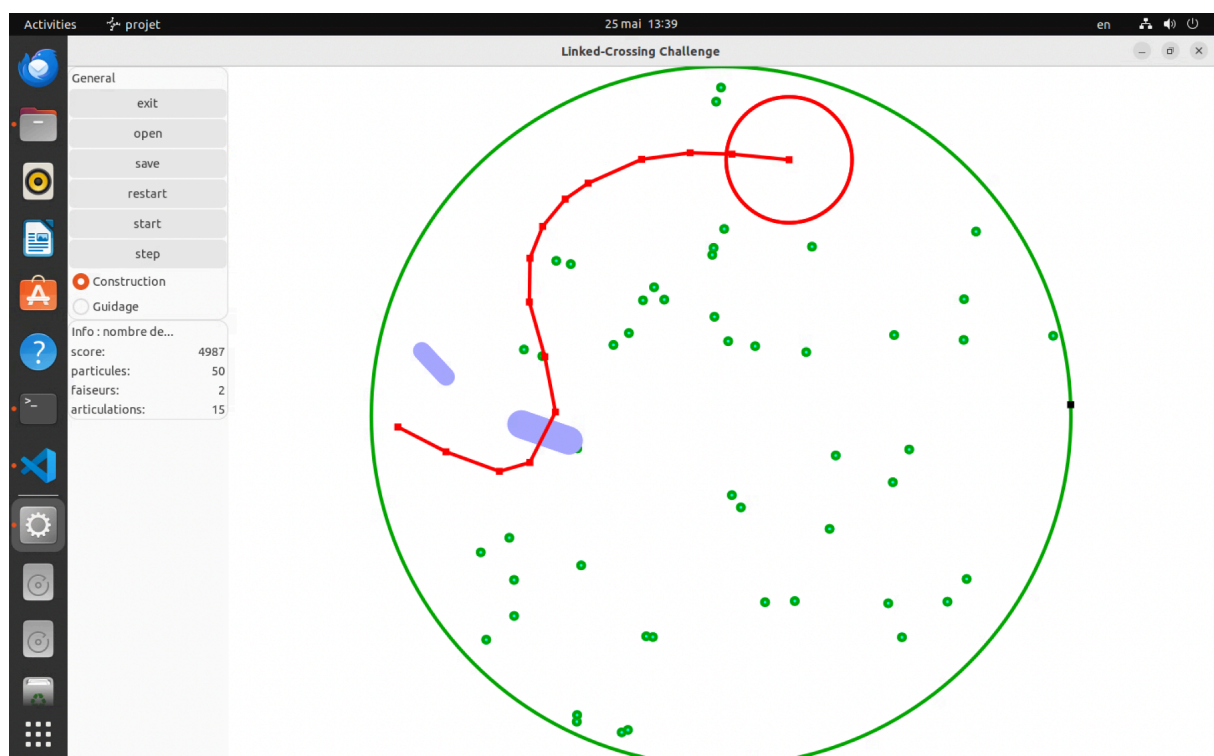
[Image 4 : Chaîne étendue montrant l'alternance construction/guidage]



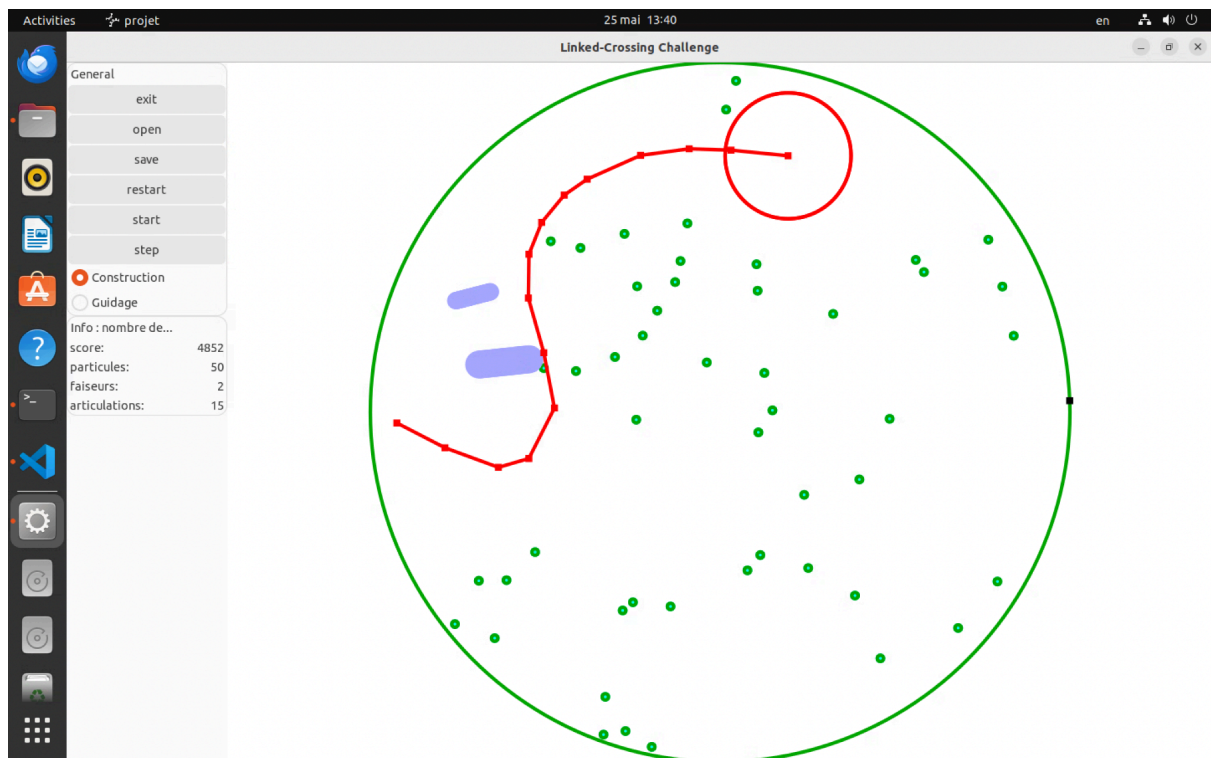
Démontre le workflow complet de construction : mécaniques de capture de particules, croissance de chaîne par positionnement stratégique, et utilisation efficace du mode guidage pour éviter les obstacles et cibler les particules.

t35.txt : Mécanique de Destruction de Chaîne

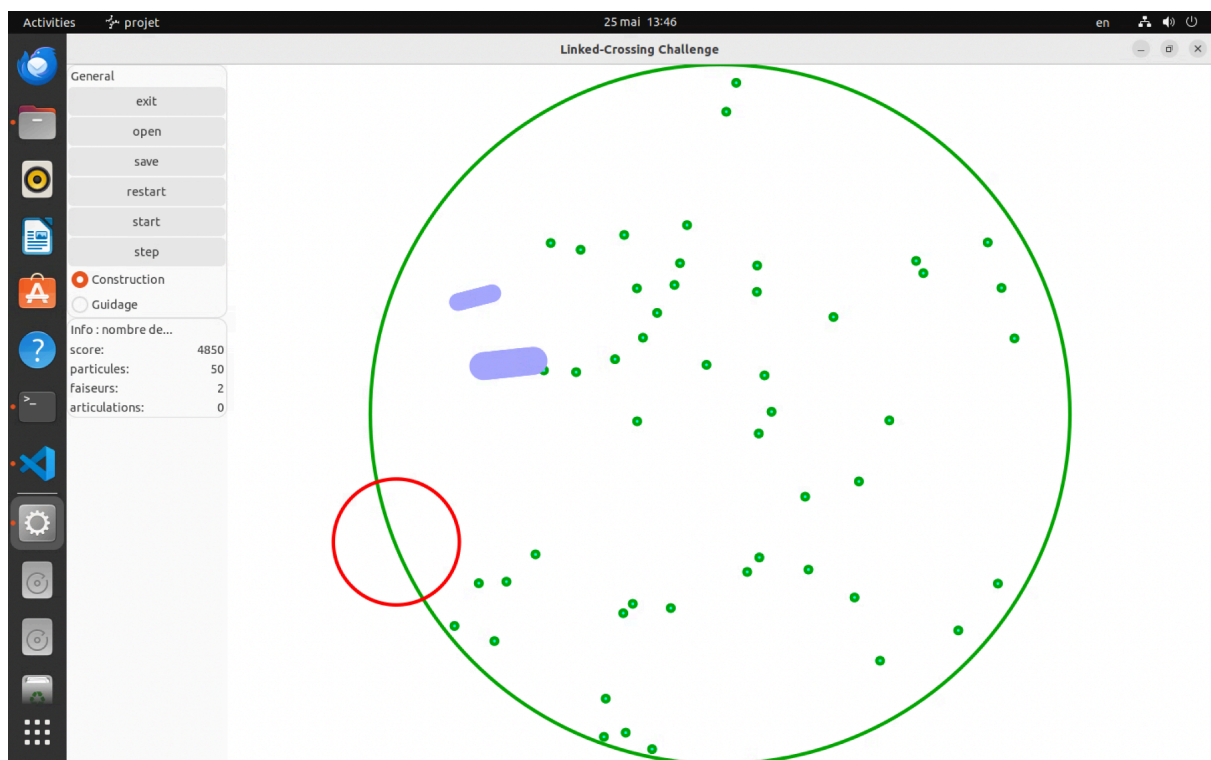
[Image 5 : État pré-collision avec chaîne établie]



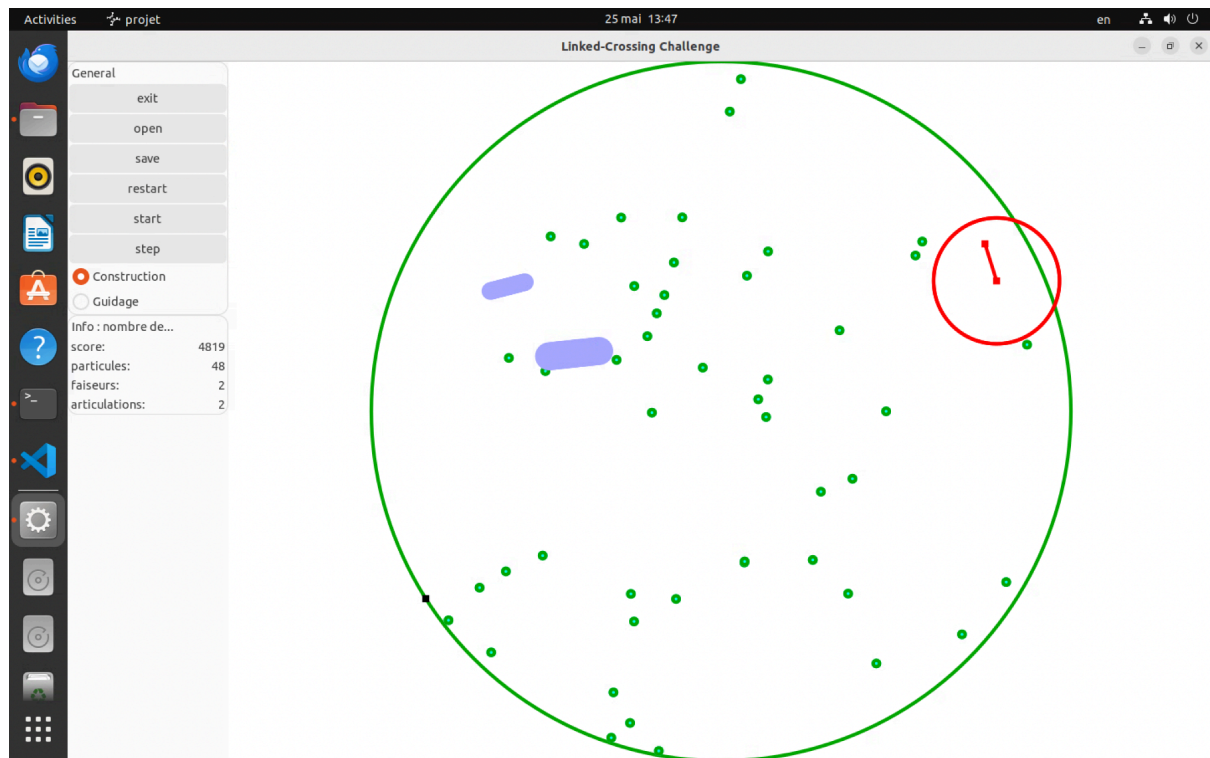
[Image 6 : Moment critique - chaîne approchant le faiseur (score 4852)]



[Image 7 : Post-collision - chaîne détruite]



[Image 8 : Jeu continue]



Montre la précision de détection de collision, destruction immédiate de la chaîne au contact du faiseur, et continuation de l'exécution du jeu post-destruction.

Contributions Individuelles

399554 : Implémentation algorithmique bas niveau - algorithme de guidage FABRIK, mathématiques de détection de collision, physique des entités (division de particules, mouvement de faiseurs), et validation de la logique de jeu centrale.

397957 : Architecture GUI et interaction utilisateur - intégration GTKmm, transformations de coordonnées souris, systèmes de retour visuel (régions de capture, indicateurs d'objectifs), et implémentation Model-View-Controller.

Approche de Développement et Évaluation

Méthodologie : Collaboration basée sur Git avec VSCode, 60% programmation en binôme, 40% travail indépendant. Développement modulaire : entités → logique de jeu → intégration GUI. Tests progressifs de la validation d'entité unique aux scénarios complexes.

Défis Principaux : Transformations du système de coordonnées entre GUI et espace de jeu (plus fréquent) ; validation des limites de l'algorithme FABRIK empêchant les articulations de sortir de l'arène (plus complexe - résolu par validation position par position).

Usage IA : Assistance modérée pour déboguer les calculs géométriques et la documentation de la bibliothèque GTKmm. Utile pour la vérification mathématique mais nécessitant une supervision d'intégration attentive.

Évaluation Technique : Implémente avec succès toutes les spécifications avec détection de collision efficace et interaction temps réel fluide. L'architecture s'adapte bien aux limites maximales d'entités. Zones d'optimisation : partitionnement spatial pour détection de collision, optimisation de chemin dans des scénarios d'obstacles complexes.

Conclusion : Implémentation robuste démontrant des principes solides de conception orientée objet et de simulation temps réel. L'architecture modulaire imposée a facilité le débogage et l'intégration de fonctionnalités tout au long du développement.